Qwen3-TTS Native: A Rust and CUDA Runtime for Streaming VoiceDesign Inference on NVIDIA DGX Spark

Luka Löhr

July 2026

Abstract

Qwen3-TTS combines description-conditioned speech generation with a low-rate, multi-codebook speech representation, but common serving paths retain a large general-purpose framework stack. This paper describes Qwen3-TTS Native, a single-process Rust and CUDA implementation of the 1.7B, 12-Hz VoiceDesign checkpoint for NVIDIA DGX Spark. The implementation covers prompt construction, autoregressive talker execution, 15-step residual-code prediction, event-ordered device-to-device token transfer, incremental neural speech decoding, bounded scheduling, and progressive HTTP delivery. The runtime intentionally excludes Python, PyTorch, SGLang, vLLM, Node.js, and voice-cloning functionality. We define a controlled, digest-bound evaluation against stock SGLang-Omni across concurrency levels one, three, and six, with two rounds per implementation and client-visible definitions of time to first audio, real-time factor, streaming cadence, memory, power, energy, and reliability. In the validated two-round comparison, Native completed 1300/1300 measured requests and stock SGLang completed 1300/1300. Combined-round aggregate RTF at B1, B3, and B6 was 0.8011, 0.6416, 0.6170 for Native and 0.4983, 0.1915, 0.1072 for stock SGLang, respectively.

1 Introduction

Modern text-to-speech systems increasingly combine a language-model-style generator with a learned neural speech codec. Qwen3-TTS follows this pattern and exposes textual VoiceDesign conditioning alongside multilingual synthesis [1]. Its 12-Hz family predicts a hierarchical set of speech codes and supports incremental waveform reconstruction. This structure is well-suited to interactive delivery, but the serving implementation determines whether model capability translates into bounded memory, low first-audio latency, and progressive output under concurrent load.

Qwen3-TTS Native is an inference-system implementation rather than a new model. It implements the pinned Qwen3-TTS-12Hz-1.7B-VoiceDesign checkpoint as a native Rust/CUDA service for a specific hardware target: NVIDIA DGX Spark with a GB10 GPU and `sm_121` machine code. It deliberately narrows the product surface to textual VoiceDesign. Reference-audio voice cloning, speaker enrollment, the speech-tokenizer encoder, other Qwen3-TTS variants, and the retired 0.6B model are out of scope.

The implementation makes four systems contributions:

1. a native execution path for the VoiceDesign talker, its 15-step code predictor, and the decoder-only neural speech tokenizer;
2. an event-ordered device-to-device handoff that keeps generated codec codes on the GPU until the decoder consumes them;
3. a fixed-capacity scheduler with request-local CUDA state, bounded PCM ownership, backpressure, cancellation, and progressive HTTP framing; and
4. a fail-closed evaluation protocol that compares an immutable native candidate with a pinned stock SGLang-Omni comparator using one external client and digest-bound raw evidence.

The present source is intentionally conservative. It documents implemented mechanisms and a complete evaluation design, but does not turn exploratory measurements into publication claims. Section 6 defines the acceptance protocol, while Section 7 remains evidence-gated.

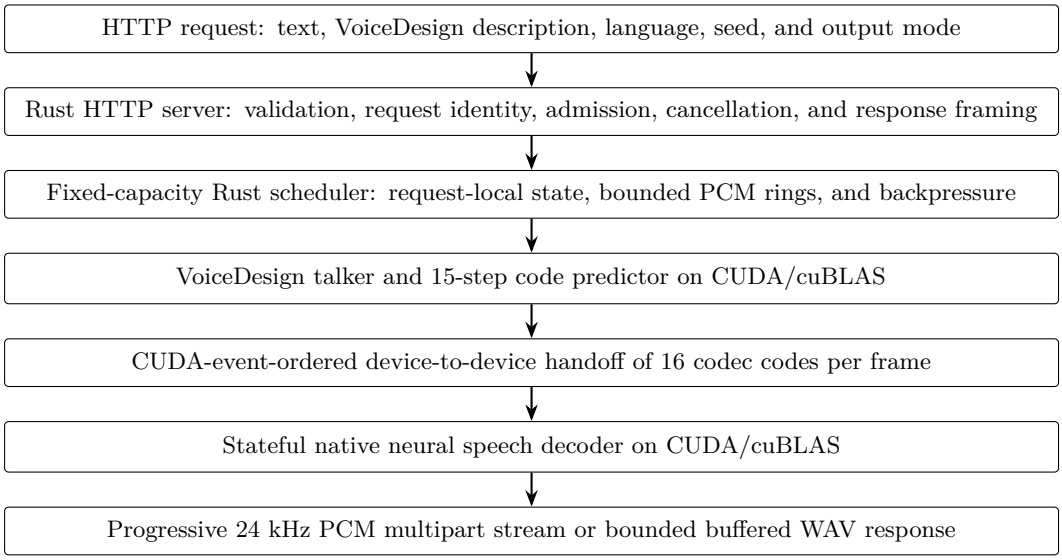

Figure 1: Implemented online path. Immutable model weights are shared, while mutable request state and CUDA execution resources are request-local. The diagram is a vector TikZ source and encodes no benchmark result.

2 Related Work

Qwen3-TTS. The Qwen3-TTS technical report presents a family of multilingual, description-controllable text-to-speech models and two speech-tokenizer rates [1]. The 12-Hz design uses residual vector-quantized speech tokens: a backbone predicts the zeroth codebook and a multi-token prediction module produces the residual codebooks. The report evaluates model quality and streaming efficiency under its own inference environment. Our work neither changes the trained model nor reproduces those model-quality claims. It studies a separate systems question: how to execute the released 1.7B VoiceDesign checkpoint through a purpose-built Rust/CUDA path on DGX Spark.

SGLang and the stock comparator. SGLang introduced a programming and runtime system for structured language model workloads, including runtime optimizations such as RadixAttention [2]. The comparison in this paper uses the separately versioned SGLang-Omni 0.1.0 Qwen3-TTS serving path, not the experiments reported in the original SGLang paper. Its upstream source commit, container base, dependency inventory, compatibility patch, and launch configuration are pinned in the benchmark evidence. The comparator remains labeled *stock* only when packaging changes do not alter scheduling, model execution, codec cadence, or streaming behavior.

Scope of comparison. The paper compares serving implementations, not trained model quality. Both subjects use the same checkpoint repository, immutable revision, ordered input workload, language values, VoiceDesign instructions, seeds, portable sampling controls, duration ceiling, client, host, and audio measurement boundary. Implementation-specific model artifacts are recorded separately because the native runtime packages only decoder inference material whereas the stock stack may retain a broader speech-tokenizer artifact.

3 System Design

Figure 1 shows the complete online path. A single process owns HTTP validation, scheduling, model execution, neural decoding, and response delivery. The server loads one shared engine, creates a fixed-capacity scheduler, and completes a real one-frame native warm-up before binding its listener. Readiness therefore establishes that the model and end-to-end native pipeline have produced a valid PCM packet; it is not merely a process-liveness signal.

3.1 Admission and ownership

The canonical synthesis endpoint validates bounded JSON before attempting engine admission. Text, textual VoiceDesign instruction, language, duration, sampling settings, and response mode are checked at this boundary.

Each admitted request receives a unique identity and owns its cancellation state, generation session, decoder session, CUDA stream and events, cuBLAS handle, random state, PCM buffers, and scheduler slot. Immutable model weights are shared across sessions.

The scheduler supports one through six active slots. Pending starts can be coalesced for at most 2 ms, but capacity is fixed and exhaustion is explicit. Each request has three reusable PCM buffers. A request advances only when a buffer and queue position are available, so a slow client applies backpressure instead of causing unbounded audio allocation.

3.2 Packet cadence

One codec frame represents 1,920 output samples, or 80 ms at 24 kHz. The first decoder step requests one frame to minimize first-audio latency. Subsequent steps request up to four frames and therefore produce at most 7,680 samples per packet. This policy bounds buffering without forcing every later step to pay the overhead of a one-frame packet.

3.3 Lifecycle and failure handling

Natural codec end-of-sequence closes a request after all owned PCM has been delivered. Cancellation is propagated through the scheduler to native state, and resource retirement is bounded. Model-contract, ABI, CUDA, packet-order, and state-transition failures are surfaced instead of silently falling back to a different implementation. The process does not attempt unsafe in-process recovery of a stuck CUDA session.

4 Native Implementation

4.1 Artifact boundary

The runtime consumes the immutable VoiceDesign snapshot at revision `5ecdb67327fd37bb2e042aab12ff7391903235d3`. Build-time and startup checks validate configuration fields, tensor metadata, artifact hashes, and the native ABI before serving traffic. The production image carries the VoiceDesign weights and decoder-only speech-tokenizer material required for inference. It excludes the tokenizer encoder, Base and CustomVoice checkpoints, reference-audio functionality, and 0.6B weights.

4.2 Talker and residual predictor

The Rust model layer parses configuration and tokenizer artifacts, constructs the VoiceDesign prompt from text, instruction, and official language name, and owns the request lifecycle. Shared weights are uploaded once. Each session retains independent key-value caches, workspaces, counters, random state, CUDA stream, and cuBLAS handle.

For each codec frame, the talker predicts the semantic code and the native 15-step predictor produces the residual codebook values, yielding 16 codes per frame. Sampling state is local to the request and is initialized from the effective seed. Predictor and talker controls remain distinct; the predictor's repetition penalty is fixed by the implemented model contract.

4.3 Device-to-device handoff

The production ABI avoids a codec-token round trip through host memory. The talker exposes its device-resident frame and records a producer-ready CUDA event. A non-blocking staging stream waits on that event, copies each frame into a bounded device packet, and records completion. The decoder waits on the packet-ready event and records a consumed event before the staging slot can be reused. Tickets and device identity are checked so stale or cross-device state cannot be accepted accidentally.

4.4 Incremental decoder

The decoder shares immutable weights but gives each active request its own incremental neural state, stream, cuBLAS handle, events, histories, and bounded PCM staging. CUDA kernels and cuBLAS execute the decoder path and clamp output into signed 16-bit PCM. The current implementation uses non-blocking streams, events, custom kernels, and cuBLAS. It does not claim CUDA Graph capture or replay.

4.5 Production image

The runtime container targets `linux/arm64` and real `sm_121` SASS without a PTX fallback. The final stage contains CUDA runtime and the required cuBLAS families but excludes compilers, development packages, Python, Node.js, PyTorch, SGLang, vLLM, TensorRT, cuDNN, NPP, cuSPARSE, and NCCL. It runs as an unprivileged user and is designed for a read-only root filesystem with dropped Linux capabilities.

5 Streaming API

The primary endpoint is `POST /v1/voice-design/speech`. A request contains non-empty UTF-8 text, a natural-language VoiceDesign description, an optional language (default `auto`), an optional seed, a bounded maximum duration, and an output mode. The endpoint accepts VoiceDesign instructions only; a voice name, speaker embedding, reference sample, or cloning identifier is not part of the schema.

Progressive responses use `multipart/mixed`. The first part is a JSON start event, subsequent binary parts contain 24,000-Hz mono signed 16-bit little-endian PCM, and exactly one JSON end or error event terminates the application stream. Multipart boundaries define application packets; HTTP/1.1 chunks or HTTP/2 DATA frames do not. A separate buffered mode returns a bounded RIFF/WAVE object after synthesis completes.

The service also exposes liveness, readiness, capability discovery, bounded request cancellation, and prompt-free Prometheus counters. A conservative compatibility endpoint returns buffered WAV but is not presented as a complete implementation of any third-party audio API.

The standalone process does not terminate TLS or implement identity. A public deployment therefore requires an authenticated and rate-limited proxy with bounded request and response timeouts. Normal service logs and metrics omit text, voice descriptions, language values, request identifiers, and generated audio; surrounding proxies and clients require their own payload-retention controls.

6 Evaluation Methodology

The evaluation is a controlled comparison of the native release candidate and the pinned stock SGLang-Omni comparator on one DGX Spark. It is designed to answer serving-system questions about latency, throughput, streaming cadence, resource use, and reliability. It does not evaluate perceptual speech quality.

6.1 Subjects and workload

The production matrix contains 12 runs: two implementations, concurrency profiles B1, B3, and B6, and two rounds. Each run performs 24 unmeasured warm-up requests followed by at least 200 successfully completed measured requests. Only one subject may use CUDA during a run. Round one executes stock SGLang before Native; round two reverses that order and executes Native before stock SGLang. The Native blocks on either side of the round boundary remain consecutive, avoiding one unnecessary model reload while preserving the required round-level order reversal.

Both subjects receive the exact same ordered JSONL workload, model repository and revision, language, text, VoiceDesign description, seed, portable sampling controls, streaming mode, and 20.48-s duration ceiling. Workload identity is bound by SHA-256, including the ordered row seeds. The external Rust HTTP client and localhost transport are shared.

Native completion is accepted only with explicit natural end-of-sequence, `finish_reason=stop`, and no length limit. The stock raw-PCM endpoint does not expose a finish reason; its natural-EOS and length-limit fields must remain unknown. As a conservative boundary gate, an accepted stock response must be strictly shorter than 489,600 samples and 20.40 s. Passing that gate excludes a known length-boundary case but does not prove natural EOS.

6.2 Client-visible metrics

The client records a monotonic start time immediately before writing each request. Time to first audio (TTFA) ends when the first non-empty decoded PCM payload byte reaches the client. Headers, multipart JSON, WAV headers, framing, and zero-length transport chunks do not count as audio.

For request i, with wall time w_i and decoded audio duration a_i,

Table 1: Per-run performance results. TTFA is measured at the client-visible first non-empty PCM byte. Aggregate RTF is scenario wall time divided by the total duration of audio generated by successful requests.

Engine	Round	Profile	Concurrency	Successes	TTFA p50 (ms)	TTFA p95 (ms)	Aggregate RTF
Native	1	B1	1	200	86.364	93.889	0.7997
Native	1	B3	3	210	200.305	216.809	0.6414
Native	1	B6	6	240	393.430	406.045	0.6174
Stock SGLang	1	B1	1	200	2256.829	2691.506	0.4971
Stock SGLang	1	B3	3	210	2294.895	2720.932	0.1864
Stock SGLang	1	B6	6	240	2423.521	2873.489	0.1024
Native	2	B1	1	200	86.730	95.582	0.8026
Native	2	B3	3	210	201.398	215.551	0.6417
Native	2	B6	6	240	391.702	405.384	0.6166
Stock SGLang	2	B1	1	200	2212.903	2703.914	0.4995
Stock SGLang	2	B3	3	210	2210.979	2814.635	0.1969
Stock SGLang	2	B6	6	240	2349.131	3145.884	0.1124

$$\mathrm{RTF}_i = \frac{w_i}{a_i}. \tag{1}$$

For a concurrent scenario with elapsed wall time W and successful requests S, the throughput-oriented aggregate real-time factor is

$$\mathrm{RTF}_{\mathrm{aggregate}} = \frac{W}{\sum_{i \in S} a_i}. \tag{2}$$

The report keeps this quantity distinct from $\sum_i w_i / \sum_i a_i$, which is a summed-request-wall latency view. It also reports successful and attempted request throughput, first-to-final PCM streaming window, packet arrivals, inter-packet gaps, response validity, timeouts, cancellations, malformed responses, and explicit termination fields.

6.3 Resource and energy measurement

Timestamped host, cgroup, NVIDIA, and power telemetry is sampled every 100 ms; a run fails if any required stream has an observed gap greater than 200 ms. Measured-phase boundaries exclude warm-up work from performance and energy reduction. Process RSS and GPU-visible unified memory are reported separately and are not summed as independent physical memory. Energy is integrated from the timestamped board-power series after subtraction of a separately measured idle baseline, then normalized per generated audio minute. An unavailable sensor remains unavailable rather than becoming an estimate.

6.4 Fail-closed publication gate

A run is excluded if another CUDA process is present, evidence is missing or digest-mismatched, the workload or normalized sampling differs between subjects, the success minimum is not met, Native completion is not natural EOS, Stock EOS is imputed, telemetry cadence fails, or the comparator contains an undisclosed execution change. Setup failures remain in the audit trail but do not enter distributions. Every accepted evidence path is relative to the manifest root and bound by byte length and SHA-256.

7 Results

Across the six validated runs per engine, Native completed 1300/1300 measured requests and stock SGLang completed 1300/1300. For B1, combined-round TTFA p95 was 94.768 ms for Native and 2694.890 ms for stock SGLang; aggregate RTF was 0.8011 and 0.4983, respectively. For B3, combined-round TTFA p95 was 216.143 ms for Native and 2793.626 ms for stock SGLang; aggregate RTF was 0.6416 and 0.1915, respectively. For B6, combined-round TTFA p95 was 405.617 ms for Native and 2929.505 ms for stock SGLang; aggregate RTF was 0.6170 and 0.1072, respectively. Native recorded natural EOS for 1300/1300 successful requests; stock SGLang retained unknown EOS for 1300/1300 successful requests.

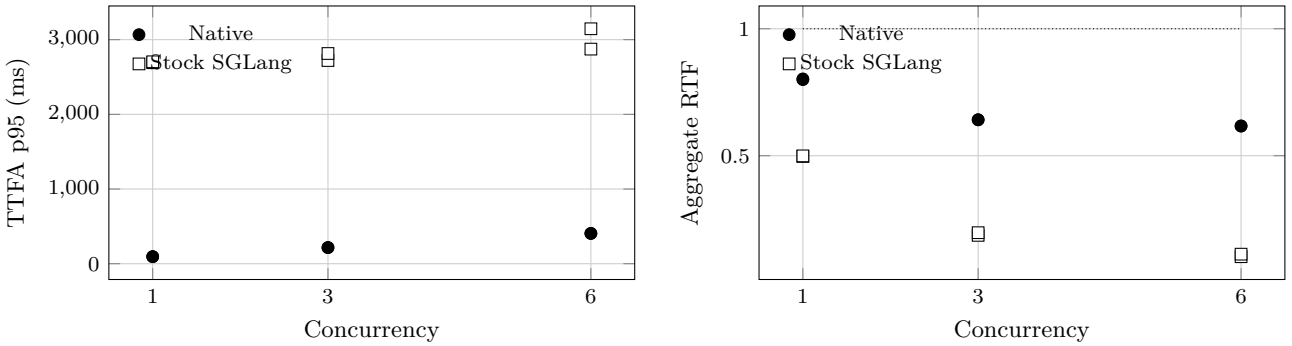

Figure 2: Per-round client-visible TTFA p95 and aggregate real-time factor at concurrency 1, 3, and 6. Filled circles denote Native; open squares denote stock SGLang. The dotted RTF line marks an RTF of 1.0. No interpolated or synthetic measurements are permitted.

Table 2: Measured-phase resource and energy results. GPU-visible unified memory is reported separately from process RSS and is not added to it.

Engine	Round	Profile	Peak RSS (GiB)	GPU-visible memory (GiB)	Mean power (W)	Energy per audio minute (J)
Native	1	B1	1.860	4.747	32.288	872.845
Native	1	B3	1.863	4.967	36.972	868.377
Native	1	B6	1.869	5.293	37.296	824.306
Stock SGLang	1	B1	4.890	101.401	36.701	711.102
Stock SGLang	1	B3	4.785	101.401	32.633	196.744
Stock SGLang	1	B6	4.791	101.417	33.446	114.430
Native	2	B1	1.869	5.094	32.861	846.716
Native	2	B3	1.869	5.174	36.739	854.261
Native	2	B6	1.870	5.293	37.164	829.147
Stock SGLang	2	B1	4.997	101.024	36.492	690.597
Stock SGLang	2	B3	5.023	101.108	32.008	222.728
Stock SGLang	2	B6	5.033	101.278	33.326	127.742

The interpretation preserves both rounds, tail latency, reliability counts, and the distinction between progressive and completion-buffered delivery. Aggregate RTF under concurrency is not replaced by mean per-request RTF. Process RSS and GPU-visible unified memory remain separate measures. Registry compressed-size comparisons are permitted only when both values are backed by registry evidence; local unpacked Docker size is never substituted.

8 Limitations

The implementation and measurements target one platform: `linux/arm64` on NVIDIA DGX Spark with `sm_121`. They do not establish portability or performance on x86-64, another GPU architecture, a multi-GPU server, or a cloud accelerator. Two rounds on one host expose run-to-run variation and reverse subject order, but they cannot characterize population-level hardware variance or eliminate all persistent host-state and thermal effects. Idle baselines and one-at-a-time CUDA isolation constrain these effects without proving that they are absent.

The runtime supports only the pinned 1.7B, 12-Hz VoiceDesign checkpoint. It does not implement voice cloning, reference-audio conditioning, speaker enrollment, Base or CustomVoice checkpoints, or the 0.6B model. Explicit language identifiers are limited to Chinese, English, Japanese, Korean, German, French, Russian, Portuguese, Spanish, and Italian, plus automatic selection. Unsupported languages are rejected rather than advertised based on anecdotal output.

The benchmark measures serving behavior, not speech naturalness, intelligibility, instruction adherence, accent, or listener preference. A faster result would not by itself demonstrate better audio. Conversely, the same released checkpoint does not guarantee bit-identical waveforms across two different implementations unless a separate parity test establishes that property.

The native and stock HTTP paths do not expose identical termination semantics. Native reports natural EOS explicitly; stock raw PCM does not. The evaluation therefore preserves Stock EOS as unknown even when transport completion and a conservative length gate succeed. Finally, the current native path does not use CUDA

Table 3: Container and model footprint. Registry compressed size is reported only when digest-bound registry evidence exists; otherwise it is shown as N/A. Local unpacked size is never substituted for registry size.

Engine	Local unpacked (GiB)	Registry compressed (GiB)	Model parameters	Model weights (GiB)
Native	4.834	N/A	2030999489	3.783
Stock SGLang	27.319	N/A	2087233793	4.206

Table 4: Digest-bound artifact identity. Registry values not captured by the controlled comparison are reported as `N/A`.

Record	Value
Evidence state	VALIDATED PRODUCTION
Native Git commit	`d5457a6ca3fc87df2190aa8ecd0fb36a5215a632`
Stock Git commit	`8e272bcb8832ef1a3865ec48255d36f4871ec885`
Native local image ID	`sha256:b716553161f7b5de41d1cad9ca512d213fad72b282a368194778aad30cf9c000`
Native OCI digest	N/A
Stock local image ID	`sha256:9930cc808840e9bac03577f664fda8b44735eb1e531c56fec8e9ad14c9eb41d2`
Stock OCI digest	N/A
Shared model revision	`5ecdb67327fd37bb2e042aab12ff7391903235d3`
Native model-artifact evidence SHA-256	`76728aaeaaec6ce3297a96def800c138efd247a75b9771cb73ac7438200365f5`
Stock model-artifact evidence SHA-256	`f28c7898475547ad2cee7ed60431439e1709139e2daa94a83c576696f00ab715`
Workload SHA-256	`56a85f56d7473529a9287cac1981dc851a29cecf18c48e46e7d3974e1aa2e0cb`
Evidence-manifest SHA-256	`1ecc13f96a07bd9642a93a810c1d4e6821b450cbc1a6679d0ef60552b5994682`
Evidence timestamp	`2026-07-17T23:18:36Z`

Graph capture or replay, so this paper makes no claim about their effect.

9 Reproducibility and Artifact Identity

The release record binds source, containers, model material, workload, raw measurements, telemetry, and rendered reports. Local Docker image IDs and unpacked sizes are distinct from registry manifest digests and compressed sizes; neither is inferred from the other. Native and stock model-artifact inventories are also recorded independently even though both share the same repository revision.

The public code location is `https://github.com/luka-loehr/qwen3-tts-native`. The immutable v0.1.0 release record binds the annotated source tag to the exact released OCI digest and publishes the English LaTeX source, bibliography, vector figures, generated plot data, final paper PDF, benchmark report PDF, evidence manifest, and checksums. A registry digest shown as `N/A` in Table 4 means that the controlled comparison captured only the local image identity; it never authorizes deployment by a mutable tag. Large raw evidence may be hosted separately when the repository cannot carry it. Any publication record for that bundle is external to schema v1.2 and must not be inferred from a local path or mutable URL.

Reproduction requires a clean pull by immutable image digest, an idle supported GPU, the documented hardened run command, a successful full-pipeline readiness check, and the external workload client. Reproducers should validate the manifest before generating tables or plots and should report any hardware, driver, power-mode, workload, or comparator difference rather than presenting it as the original experiment.

10 Ethics and Broader Impacts

Description-conditioned speech can improve accessibility, multilingual interfaces, educational tools, and local control over speech infrastructure. It can also be used to create misleading or harassing audio. Removing reference-audio cloning from this runtime narrows one misuse path but does not make generated speech inherently benign: a textual description may still seek to evoke a recognizable person, role, or demographic stereotype.

Deployers should require authorization, rate limits, abuse monitoring, appropriate disclosure that audio is synthetic, and a retention policy for input text and generated audio. Applications involving an identifiable person require consent and applicable legal review. Provenance or watermarking should be evaluated where the deployment context supports it; this runtime does not claim to add an inaudible watermark.

Benchmark publication also has an integrity obligation. Failed trials, contamination, unsupported languages, unknown EOS state, unavailable sensors, and negative results must remain visible. Energy measurements from one device should not be generalized into a lifecycle environmental claim without a broader accounting boundary.

11 Conclusion

This paper describes a complete native Rust/CUDA serving path for the pinned Qwen3-TTS 1.7B VoiceDesign checkpoint on NVIDIA DGX Spark. The system combines request-local model sessions, event-ordered device token handoff, incremental neural decoding, bounded scheduling, backpressure, cancellation, and progressive PCM delivery in one process and one production image.

The accompanying evaluation design makes implementation comparisons auditable: one external client, one ordered workload, two rounds, three concurrency levels, explicit termination semantics, measured-phase telemetry, and digest-bound raw evidence.

The accepted evidence contains 2600 successful measured responses across twelve runs. For B1, combined-round TTFA p95 was 94.768 ms for Native and 2694.890 ms for stock SGLang; aggregate RTF was 0.8011 and 0.4983, respectively. For B3, combined-round TTFA p95 was 216.143 ms for Native and 2793.626 ms for stock SGLang; aggregate RTF was 0.6416 and 0.1915, respectively. For B6, combined-round TTFA p95 was 405.617 ms for Native and 2929.505 ms for stock SGLang; aggregate RTF was 0.6170 and 0.1072, respectively. The Native series preserved explicit natural-EOS classification, while stock SGLang retained unknown EOS as required by its transport contract.

References

[1] Hangrui Hu, Xinfa Zhu, Ting He, Dake Guo, Bin Zhang, Xiong Wang, Zhifang Guo, Ziyue Jiang, Hongkun Hao, Zishan Guo, Xinyu Zhang, Pei Zhang, Baosong Yang, Jin Xu, Jingren Zhou, and Junyang Lin. Qwen3-TTS technical report. *arXiv preprint arXiv:2601.15621*, 2026.

[2] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. SGLang: Efficient execution of structured language model programs. *arXiv preprint arXiv:2312.07104*, 2023.

A Artifact, API, and Release Acceptance

This appendix summarizes the normative acceptance boundary for the software artifact and its public API. It introduces no additional measurement and does not replace the digest-bound evidence described in Section 6.

A.1 Artifact boundary

An accepted release is built from one reviewed, clean `main` commit and identifies the deployed image by its complete immutable GHCR digest. A branch, local image ID, candidate tag, semantic-version tag, or `latest` alias is not a deployment identity by itself. Aliases may be published only when they resolve to the already accepted digest without rebuilding the image.

The runtime artifact is limited to the pinned Qwen3-TTS-12Hz-1.7B-VoiceDesign checkpoint and the decoder-only speech-tokenizer material required for inference. The image embeds those weights and must run without a host model mount. Voice cloning, reference-audio conditioning, speaker enrollment, the speech-tokenizer encoder, Base and CustomVoice checkpoints, and the 0.6B model remain outside the artifact contract.

A.2 API acceptance surface

Table 5 lists the complete public endpoint surface used by release acceptance. The canonical synthesis endpoint accepts text and a natural-language VoiceDesign description. Progressive output uses `multipart/mixed` with 24 kHz mono signed 16-bit PCM; buffered output uses a bounded RIFF/WAVE response.

Table 5: Public endpoints covered by the release acceptance boundary.

Endpoint	Acceptance purpose
`GET /health/live`	Process and HTTP-loop liveness.
`GET /health/ready`	Shared-engine readiness after the complete native warm-up.
`GET /v1/capabilities`	Effective VoiceDesign-only languages, formats, and limits.
`POST /v1/voice-design/speech`	Canonical progressive PCM or buffered WAV synthesis.
`POST /v1/audio/speech`	Narrow buffered-WAV compatibility behavior.
`DELETE /v1/requests/{request-id}`	Bounded cancellation of an admitted request.
`GET /metrics`	Prompt-free request and engine-health metrics.

API acceptance requires readiness, progressive delivery before synthesis completion, a valid buffered WAV response, cancellation, bounded shutdown and restart behavior, and payload-free operational metrics. The compatibility endpoint is not treated as a complete implementation of a third-party audio API. Public deployment additionally requires an authenticated, rate-limited TLS-terminating proxy because the native service does not provide those edge controls.

A.3 Evidence and reproducibility gates

Publication requires the complete schema-v1.2 comparison: Native and stock SGLang at B1, B3, and B6 in two rounds, with the shared workload and client, checksum-inventoried raw evidence, the declared warm-up minimum, and at least 200 successful measured requests in every cell. Native successes require explicit natural end-of-sequence. Stock SGLang termination remains unknown and must pass the documented exclusive length-boundary gate without being relabeled as natural EOS. Telemetry must retain the documented cadence, measured-phase boundaries, and zero-competing-CUDA-process requirement.

The production manifest, benchmark report, paper data, figures, and tables must all derive from that same validated evidence root. Release acceptance then binds the clean source commit, embedded model identity, immutable OCI digest, SBOM, provenance, vulnerability and secret scans, keyless signature, anonymous empty-store pull, and digest-specific GPU checks. The semantic-version and `latest` aliases are promoted only after those gates pass and must continue to resolve to the accepted digest.

A reproducer should record the source tag and commit, immutable image digest, model revision, evidence-manifest SHA-256, workload SHA-256, hardware, driver, and CUDA versions. The manifest must be validated before regenerating the report or paper data. Any change to hardware, workload, comparator, sampling, duration, or termination semantics must be disclosed as a different experiment rather than presented as a reproduction of the release result.